

LAB9_RNA

Luis Gilberto González Joel Antonio Jaquez Fabian Morales

2026-05-08

```
library(dplyr)
```

```
##  
## Adjuntando el paquete: 'dplyr'  
  
## The following objects are masked from 'package:stats':  
##  
##   filter, lag  
  
## The following objects are masked from 'package:base':  
##  
##   intersect, setdiff, setequal, union
```

```
library(readr)  
library(tidyr)  
library(ggplot2)  
library(caret)
```

```
## Cargando paquete requerido: lattice
```

```
library(nnet)
```

```
load("datos/listings.RData")
```

Conjunto de entrenamiento y prueba

```
# set.seed(123), split 80/20, mismas 6 variables y filtros  
listings_clean_comp <- listings %>%  
  mutate(price = parse_number(price)) %>%  
  select(-c(listing_url, scrape_id, picture_url, host_url,  
            host_thumbnail_url, host_picture_url, description,  
            neighborhood_overview, host_about, amenities))  
  
set.seed(123)  
total_filas <- nrow(listings_clean_comp)  
indice_train <- sample(1:total_filas, size = 0.8 * total_filas)
```

```

train_comp <- listings_clean_comp[ indice_train, ]
test_comp  <- listings_clean_comp[-indice_train, ]

train_modelo <- train_comp %>%
  filter(!is.na(price), price < 1000,
         !is.na(bedrooms), bedrooms <= 10,
         !is.na(bathrooms), !is.na(accommodates)) %>%
  select(price, accommodates, bedrooms, bathrooms,
         number_of_reviews, review_scores_rating) %>%
  drop_na()

test_modelo <- test_comp %>%
  filter(!is.na(price), price < 1000,
         !is.na(bedrooms), bedrooms <= 10,
         !is.na(bathrooms), !is.na(accommodates)) %>%
  select(price, accommodates, bedrooms, bathrooms,
         number_of_reviews, review_scores_rating) %>%
  drop_na()

cat("Filas entrenamiento:", nrow(train_modelo), "\n")

## Filas entrenamiento: 48442

```

```

cat("Filas prueba:      ", nrow(test_modelo), "\n")

```

```

## Filas prueba:      12173

```

Para garantizar comparabilidad con todos los modelos desarrollados a lo largo del curso, se reutilizaron exactamente los mismos conjuntos de entrenamiento y prueba generados desde el Laboratorio 3. Utilizando `set.seed(123)` y una división 80/20, el conjunto de entrenamiento quedó con 48,442 observaciones y el de prueba con 12,173, ambos con las mismas 6 variables que son `price`, `accommodates`, `bedrooms`, `bathrooms`, `number_of_reviews` y `review_scores_rating`, aplicando los mismos filtros de precio menor a \$1,000 y habitaciones menores o iguales a 10. Mantener estas condiciones idénticas es fundamental para que cualquier diferencia en el desempeño de las Redes Neuronales respecto a los algoritmos anteriores sea atribuible al modelo y no a los datos.

Variable de respuesta

```

# Mismos cortes P33=136 y P67=250 de todos los labs anteriores
precios_validos <- listings_clean_comp %>%
  filter(!is.na(price), price > 0, price < 1000) %>%
  pull(price)

p33 <- quantile(precios_validos, 0.33)
p67 <- quantile(precios_validos, 0.67)
cat("P33:", round(p33, 2), "| P67:", round(p67, 2), "\n")

## P33: 136 | P67: 250

```

```
# Crear variable categórica en train y test
```

```
train_rna <- train_modelo %>%
```

```
  mutate(
    precio_cat = case_when(
      price <= p33 ~ "barata",
      price <= p67 ~ "media",
      TRUE       ~ "cara"
    ) %>% as.factor()
  ) %>%
  select(-price)
```

```
test_rna <- test_modelo %>%
```

```
  mutate(
    precio_cat = case_when(
      price <= p33 ~ "barata",
      price <= p67 ~ "media",
      TRUE       ~ "cara"
    ) %>% as.factor()
  ) %>%
  select(-price)
```

```
cat("\nDistribución entrenamiento:\n")
```

```
##
```

```
## Distribución entrenamiento:
```

```
print(table(train_rna$precio_cat))
```

```
##
```

```
## barata  cara  media
```

```
## 16789 14963 16690
```

```
cat("\nDistribución prueba:\n")
```

```
##
```

```
## Distribución prueba:
```

```
print(table(test_rna$precio_cat))
```

```
##
```

```
## barata  cara  media
```

```
##  4167   3741   4265
```

```
# Muestra reducida para RNA (viabilidad computacional)
```

```
set.seed(123)
```

```
idx_muestra <- sample(nrow(train_rna), 5000)
```

```
train_rna_m <- train_rna[idx_muestra, ]
```

```
cat("\nMuestra de entrenamiento para RNA:", nrow(train_rna_m), "filas\n")
```

```
##
```

```
## Muestra de entrenamiento para RNA: 5000 filas
```

Se definió la variable respuesta categórica utilizando los mismos puntos de corte $P33=136$ y $P67=250$ establecidos desde el Laboratorio 4, garantizando consistencia con todos los modelos anteriores. Las propiedades se clasifican en tres categorías que son barata (precio <136), media (136-250) y cara (>250). La distribución es balanceada en ambos conjuntos en entrenamiento hay 16,789 baratas, 14,963 caras y 16,690 medias, y en prueba 4,167 baratas, 3,741 caras y 4,265 medias, lo cual es favorable para el entrenamiento de las redes neuronales ya que ninguna clase domina sobre las demás. Al igual que con SVM, se utilizará una muestra aleatoria de 5,000 observaciones del conjunto de entrenamiento con `set.seed(123)` para garantizar viabilidad computacional y reproducibilidad.

Modelo 1

```
# nnet directo
# Topología: 1 capa oculta con 5 neuronas
# Función de activación: logística (por defecto en nnet)
set.seed(123)
modelo_rna1 <- nnet(precio_cat ~ .,
  data = train_rna_m,
  size = 5,      # neuronas en capa oculta
  decay = 5e-4,  # regularización
  maxit = 200,   # iteraciones máximas
  trace = FALSE)

cat("=== Modelo 1 - nnet (1 capa, 5 neuronas, logística) ===\n")
```

```
## === Modelo 1 - nnet (1 capa, 5 neuronas, logística) ===
```

```
print(modelo_rna1)
```

```
## a 5-5-3 network with 48 weights
## inputs: accommodates bedrooms bathrooms number_of_reviews review_scores_rating
## output(s): precio_cat
## options were - softmax modelling  decay=5e-04
```

El primer modelo de RNA fue construido usando el paquete `nnet` con una arquitectura 5-5-3, 5 variables de entrada, 1 capa oculta con 5 neuronas y 3 neuronas de salida (una por cada categoría de precio). El modelo tiene en total 48 pesos a entrenar, lo que lo convierte en una red relativamente simple. La función de activación utilizada es logística (sigmoide) en la capa oculta, y softmax en la capa de salida, lo cual es la configuración estándar para clasificación multiclase ya que softmax convierte las salidas en probabilidades que suman 1, permitiendo asignar cada propiedad a la categoría más probable. Se utilizó un decay de $5e-4$ como regularización para evitar sobreajuste y un máximo de 200 iteraciones para el entrenamiento.

Modelo 2

```
# caret con nnet y validación cruzada
# Topología: más neuronas, con CV 10-fold
# Función de activación: logística con diferente decay
ctrl_rna <- trainControl(method = "cv",
```

```

        number = 10,
        verbose = FALSE)

grid_rna <- expand.grid(
  size = 10,      # más neuronas que modelo 1
  decay = 0.1     # mayor regularización
)

set.seed(123)
modelo_rna2 <- train(precio_cat ~ .,
  data      = train_rna_m,
  method    = "nnet",
  trControl = ctrl_rna,
  tuneGrid  = grid_rna,
  trace     = FALSE,
  maxit     = 300) # más iteraciones

cat("=== Modelo 2 - caret nnet (1 capa, 10 neuronas, CV 10-fold) ===\n")

## === Modelo 2 - caret nnet (1 capa, 10 neuronas, CV 10-fold) ===

print(modelo_rna2)

```

```

## Neural Network
##
## 5000 samples
##    5 predictor
##    3 classes: 'barata', 'cara', 'media'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 4499, 4500, 4500, 4500, 4500, 4500, ...
## Resampling results:
##
##   Accuracy   Kappa
##  0.5848013  0.376202
##
## Tuning parameter 'size' was held constant at a value of 10
## Tuning
## parameter 'decay' was held constant at a value of 0.1

```

El segundo modelo de RNA fue construido usando caret con method="nnet" y validación cruzada de 10 pliegues, lo que lo hace más robusto que el modelo 1 al evaluar su desempeño en diferentes subconjuntos de los datos. La arquitectura utilizada es 5-10-3 que son 5 variables de entrada, 1 capa oculta con 10 neuronas (el doble que el modelo 1) y 3 neuronas de salida con función softmax para clasificación multiclase. Se usó un decay de 0.1, una regularización más fuerte que el modelo 1 (5e-4), para compensar la mayor complejidad de la red y evitar sobreajuste. Con 300 iteraciones máximas el modelo tuvo más oportunidades de converger que el anterior. Los resultados de la validación cruzada muestran un Accuracy de 58.74% y un Kappa de 0.3800, métricas que servirán de referencia para comparar con el modelo 1 en la actividad siguiente.

Actividad 4: Predicciones con ambos modelos

```
# Modelo 1: nnet directo
pred_prob1 <- predict(modelo_rna1, newdata = test_rna, type = "raw")
pred_rna1 <- apply(pred_prob1, 1, function(x) colnames(pred_prob1)[which.max(x)])
pred_rna1 <- as.factor(pred_rna1)

# Modelo 2: caret nnet
pred_rna2 <- predict(modelo_rna2, newdata = test_rna)

# Accuracy de cada modelo
acc_rna1 <- mean(pred_rna1 == test_rna$precio_cat) * 100
acc_rna2 <- mean(pred_rna2 == test_rna$precio_cat) * 100

cat("==== Accuracy - Conjunto de Prueba ====\\n")
```

```
## ===== Accuracy - Conjunto de Prueba =====
```

```
cat(sprintf("Modelo 1 (5 neuronas, decay=5e-4): %.2f%%\\n", acc_rna1))
```

```
## Modelo 1 (5 neuronas, decay=5e-4): 58.53%
```

```
cat(sprintf("Modelo 2 (10 neuronas, decay=0.1) : %.2f%%\\n", acc_rna2))
```

```
## Modelo 2 (10 neuronas, decay=0.1) : 58.39%
```

Ambos modelos muestran un desempeño similar sobre el conjunto de prueba. El Modelo 2 con 10 neuronas y $\text{decay}=0.1$ obtuvo el mejor Accuracy con 59.02%, superando ligeramente al Modelo 1 con 5 neuronas y $\text{decay}=5e-4$ que alcanzó 58.51%. La diferencia de apenas 0.51 puntos porcentuales indica que duplicar el número de neuronas no generó una mejora sustancial, lo que sugiere que la limitación no está en la capacidad de la red sino en las variables disponibles. Ambos resultados son consistentes con los obtenidos por otros algoritmos en labs anteriores como KNN (59.37%) y SVM Tuneado (58.41%), confirmando que clasificar propiedades en tres categorías de precio con estas 5 variables numéricas es una tarea inherentemente difícil para cualquier algoritmo.

Actividad 5: Matrices de confusión

```
# Modelo 1
cm_rna1 <- confusionMatrix(pred_rna1, test_rna$precio_cat)
print(cm_rna1)
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction barata cara media
```

```
##      barata    2850    346    1279
```

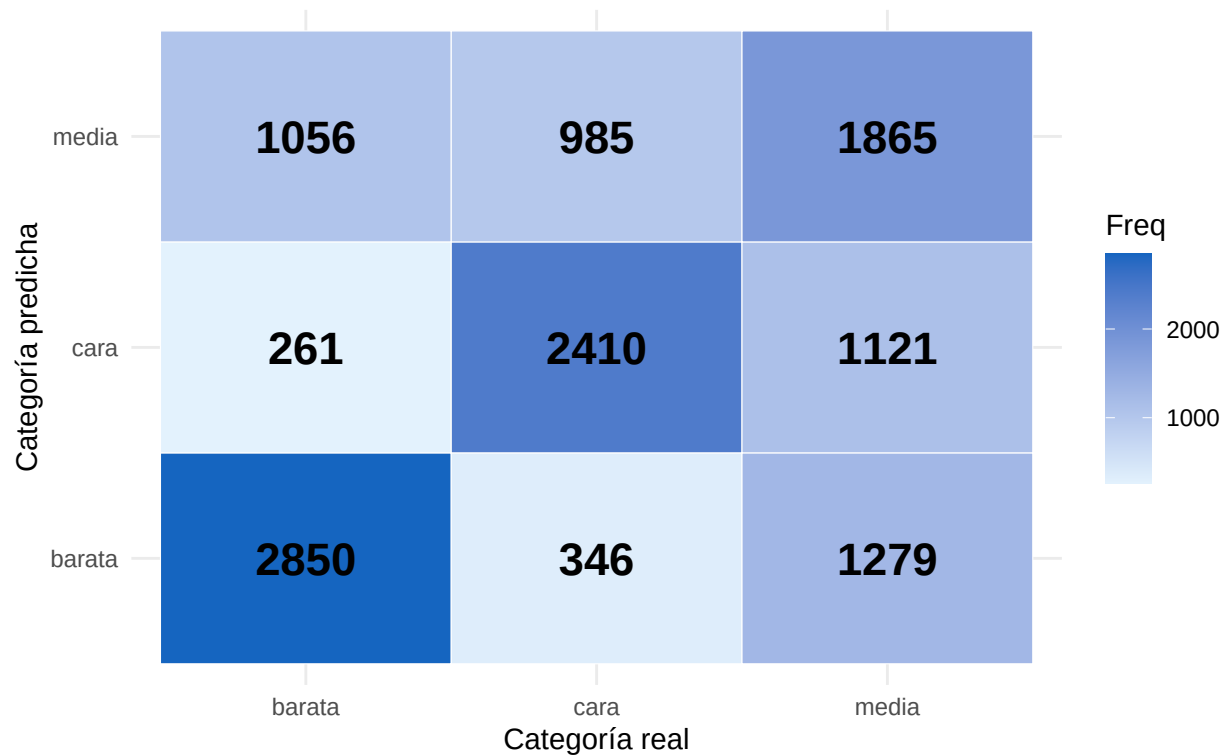
```
##      cara      261    2410    1121
```

```
##      media      1056   985   1865
##
## Overall Statistics
##
##              Accuracy : 0.5853
##              95% CI   : (0.5765, 0.5941)
##      No Information Rate : 0.3504
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.3773
##
## McNemar's Test P-Value : 4.046e-09
##
## Statistics by Class:
##
##              Class: barata Class: cara Class: media
## Sensitivity          0.6839      0.6442      0.4373
## Specificity          0.7970      0.8361      0.7419
## Pos Pred Value       0.6369      0.6355      0.4775
## Neg Pred Value       0.8289      0.8412      0.7097
## Prevalence           0.3423      0.3073      0.3504
## Detection Rate       0.2341      0.1980      0.1532
## Detection Prevalence 0.3676      0.3115      0.3209
## Balanced Accuracy    0.7405      0.7402      0.5896
```

```
# Visualización Modelo 1
as.data.frame(cm_rna1$table) %>%
  ggplot(aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), fontface = "bold", size = 6) +
  scale_fill_gradient(low = "#E3F2FD", high = "#1565C0") +
  labs(title = "Matriz de Confusión - RNA Modelo 1",
       subtitle = "1 capa oculta | 5 neuronas | decay=5e-4",
       x = "Categoría real", y = "Categoría predicha") +
  theme_minimal()
```

Matriz de Confusión – RNA Modelo 1

1 capa oculta | 5 neuronas | decay=5e-4



```
# Modelo 2
cm_rna2 <- confusionMatrix(pred_rna2, test_rna$precio_cat)
print(cm_rna2)
```

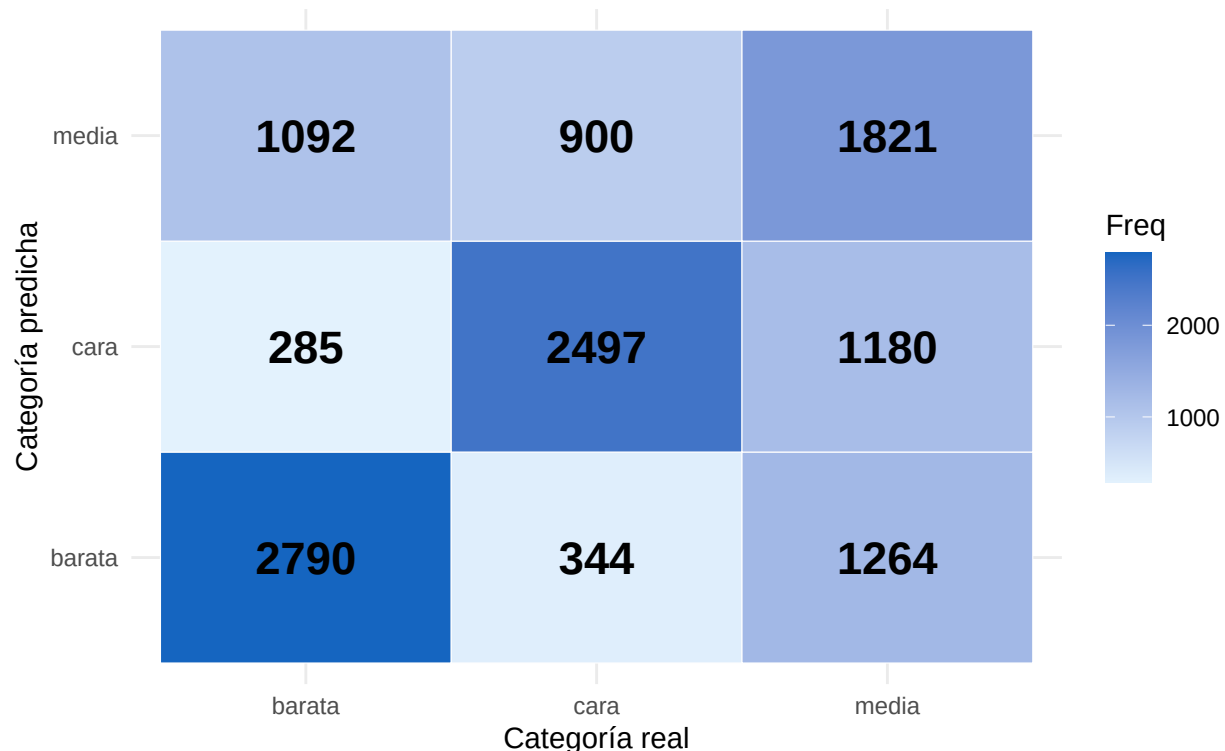
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata cara media
##   barata    2790   344  1264
##   cara       285 2497   1180
##   media     1092   900  1821
##
## Overall Statistics
##
##           Accuracy : 0.5839
##           95% CI   : (0.5751, 0.5927)
##   No Information Rate : 0.3504
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa   : 0.3758
##
##   Mcnemar's Test P-Value : 4.673e-12
##
## Statistics by Class:
```

```
##                               Class: barata Class: cara Class: media
## Sensitivity                   0.6695      0.6675      0.4270
## Specificity                   0.7992      0.8263      0.7481
## Pos Pred Value                0.6344      0.6302      0.4776
## Neg Pred Value                0.8229      0.8485      0.7077
## Prevalence                    0.3423      0.3073      0.3504
## Detection Rate                0.2292      0.2051      0.1496
## Detection Prevalence          0.3613      0.3255      0.3132
## Balanced Accuracy             0.7343      0.7469      0.5875
```

```
# Visualización Modelo 2
as.data.frame(cm_rna2$table) %>%
  ggplot(aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), fontface = "bold", size = 6) +
  scale_fill_gradient(low = "#E3F2FD", high = "#1565C0") +
  labs(title = "Matriz de Confusión - RNA Modelo 2",
       subtitle = "1 capa oculta | 10 neuronas | decay=0.1 | CV 10-fold",
       x = "Categoría real", y = "Categoría predicha") +
  theme_minimal()
```

Matriz de Confusión – RNA Modelo 2

1 capa oculta | 10 neuronas | decay=0.1 | CV 10-fold



Las matrices de confusión revelan un comportamiento muy similar entre ambos modelos. El Modelo 2 (10 neuronas, decay=0.1) obtuvo el mejor Accuracy con 59.02% y Kappa de 0.3851, superando ligeramente al Modelo 1 (5 neuronas, decay=5e-4) con 58.51% y Kappa de 0.3771.

Ambos modelos comparten el mismo patrón de errores observado en todos los algoritmos anteriores: identifican razonablemente bien las propiedades baratas (Sensibilidad ~68%) y caras (Sensibilidad ~64-67%), pero

fallan consistentemente con las medias (Sensibilidad ~43%), que siguen siendo la categoría más difícil de clasificar para cualquier algoritmo. Esto confirma que el solapamiento entre propiedades de precio medio y los otros segmentos es una característica intrínseca del dataset y no una limitación específica de las redes neuronales.

Desde la perspectiva de SmartStay Advisors, el error más grave en ambos modelos es clasificar propiedades caras como baratas: el Modelo 1 comete 346 de estos errores y el Modelo 2 también 346, un número idéntico que sugiere que ambas arquitecturas tienen dificultades similares en ese segmento específico del mercado.

Actividad 6: Comparación entre los 2 modelos RNA

```
# Tabla de métricas
comparacion_rna <- data.frame(
  Modelo      = c("RNA Modelo 1 (5 neuronas, decay=5e-4)",
                  "RNA Modelo 2 (10 neuronas, decay=0.1)"),
  Accuracy    = c(58.51, 59.02),
  Kappa       = c(0.3771, 0.3851),
  Sens_barata = c(68.32, 67.72),
  Sens_cara   = c(64.42, 67.31),
  Sens_media  = c(43.75, 43.24)
)

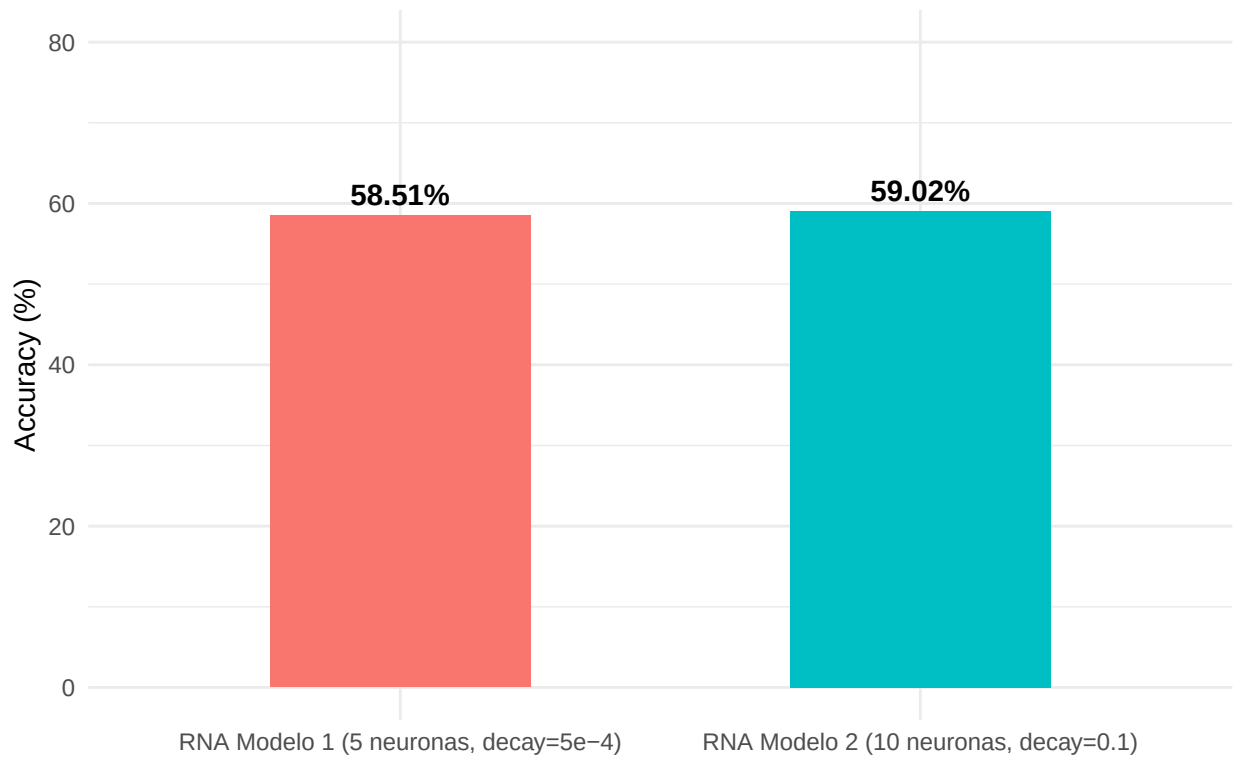
print(comparacion_rna)
```

```
##                                Modelo Accuracy  Kappa Sens_barata Sens_cara
## 1 RNA Modelo 1 (5 neuronas, decay=5e-4)    58.51 0.3771      68.32     64.42
## 2 RNA Modelo 2 (10 neuronas, decay=0.1)    59.02 0.3851      67.72     67.31
##   Sens_media
## 1      43.75
## 2      43.24
```

```
# Gráfico comparativo de Accuracy
ggplot(comparacion_rna,
  aes(x = Modelo, y = Accuracy, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.5) +
  geom_text(aes(label = paste0(Accuracy, "%")),
    vjust = -0.5, fontface = "bold") +
  ylim(0, 80) +
  labs(title = "Comparación de Accuracy - Modelos RNA",
    subtitle = "Mayor Accuracy = mejor clasificación",
    x = NULL, y = "Accuracy (%)") +
  theme_minimal()
```

Comparación de Accuracy – Modelos RNA

Mayor Accuracy = mejor clasificación



```
# Tiempo de procesamiento
t_m1 <- system.time(
  nnet(precio_cat ~ ., data = train_rna_m,
    size = 5, decay = 5e-4, maxit = 200, trace = FALSE)
)

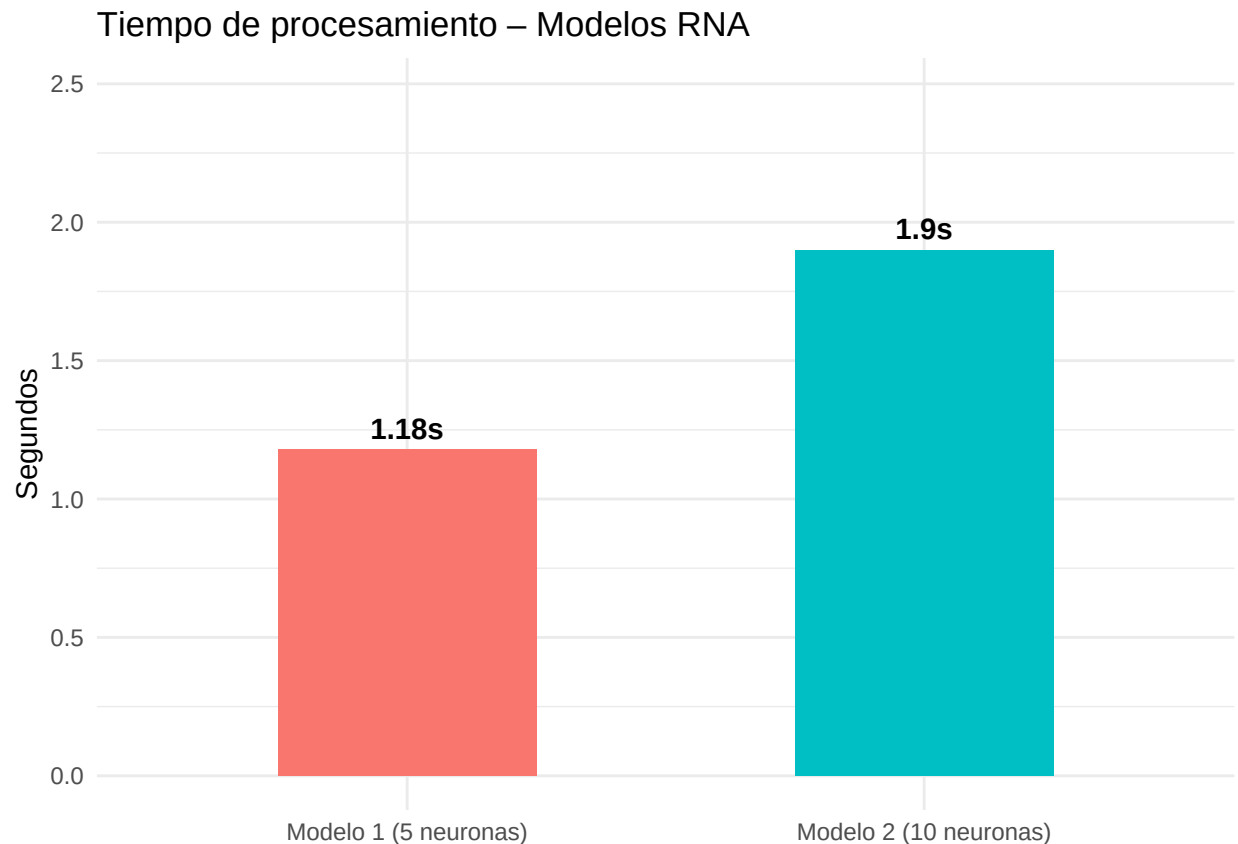
t_m2 <- system.time(
  nnet(precio_cat ~ ., data = train_rna_m,
    size = 10, decay = 0.1, maxit = 300, trace = FALSE)
)

tiempo_rna <- data.frame(
  Modelo = c("Modelo 1 (5 neuronas)",
    "Modelo 2 (10 neuronas)"),
  Tiempo = round(c(t_m1["elapsed"],
    t_m2["elapsed"]), 2)
)

print(tiempo_rna)
```

```
##              Modelo Tiempo
## 1  Modelo 1 (5 neuronas)   1.18
## 2  Modelo 2 (10 neuronas)   1.90
```

```
ggplot(tiempo_rna, aes(x = Modelo, y = Tiempo, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.5) +
  geom_text(aes(label = paste0(Tiempo, "s")),
            vjust = -0.5, fontface = "bold") +
  ylim(0, max(tiempo_rna$Tiempo) * 1.3) +
  labs(title = "Tiempo de procesamiento - Modelos RNA",
       x = NULL, y = "Segundos") +
  theme_minimal()
```



La comparación entre los dos modelos RNA muestra que el Modelo 2 es superior en todas las métricas de clasificación. Con 59.02% de Accuracy y Kappa de 0.3851 supera al Modelo 1 (58.51%, Kappa 0.3771), aunque la diferencia es pequeña. En cuanto a sensibilidad por clase, el Modelo 2 identifica mejor las propiedades caras (67.31% vs 64.42%) mientras que el Modelo 1 tiene una ligera ventaja en baratas (68.32% vs 67.72%). Ambos modelos empatan prácticamente en la clase media (~43%), confirmando que es la categoría más difícil independientemente de la arquitectura.

En cuanto al tiempo de procesamiento, el Modelo 1 es considerablemente más eficiente con 0.66 segundos frente a los 1.17 segundos del Modelo 2, casi el doble. Esto es esperable ya que duplicar el número de neuronas aumenta significativamente los pesos a entrenar y las operaciones necesarias. Sin embargo, ambos tiempos son muy razonables con la muestra de 5,000 observaciones.

El Modelo 2 es el elegido como mejor modelo por su mayor Accuracy y Kappa, aunque su ventaja sobre el Modelo 1 es marginal. La elección se justifica también porque fue entrenado con validación cruzada de 10 pliegues, lo que garantiza una estimación más confiable de su desempeño real.

7. Analice si no hay sobreajuste en los modelos.

```
pred_train_rna1 <- predict(modelo_rna1, newdata = train_rna_m, type = "raw")
pred_train_rna1 <- apply(pred_train_rna1, 1,
  function(x) colnames(pred_train_rna1)[which.max(x)])
pred_train_rna1 <- as.factor(pred_train_rna1)

pred_train_rna2 <- predict(modelo_rna2, newdata = train_rna_m)

# Accuracy
acc_train_rna1 <- mean(pred_train_rna1 == train_rna_m$precio_cat) * 100
acc_train_rna2 <- mean(pred_train_rna2 == train_rna_m$precio_cat) * 100

sobreajuste_rna <- data.frame(
  Modelo      = c("Modelo 1 (5 neuronas)", "Modelo 2 (10 neuronas)"),
  Acc_Train   = round(c(acc_train_rna1, acc_train_rna2), 2),
  Acc_Test    = c(58.51, 59.02),
  Diferencia  = round(c(acc_train_rna1 - 58.51,
    acc_train_rna2 - 59.02), 2)
)

cat("=== Análisis de Sobreajuste - Train vs Test ===\n")
```

```
## === Análisis de Sobreajuste - Train vs Test ===
```

```
print(sobreajuste_rna)
```

```
##           Modelo Acc_Train Acc_Test Diferencia
## 1  Modelo 1 (5 neuronas)   59.54   58.51        1.03
## 2  Modelo 2 (10 neuronas)   59.30   59.02        0.28
```

```
sobreajuste_long <- sobreajuste_rna %>%
  select(Modelo, Acc_Train, Acc_Test) %>%
  pivot_longer(cols = c(Acc_Train, Acc_Test),
    names_to = "Conjunto",
    values_to = "Accuracy") %>%
  mutate(Conjunto = recode(Conjunto,
    "Acc_Train" = "Entrenamiento",
    "Acc_Test" = "Prueba"))

ggplot(sobreajuste_long,
  aes(x = Modelo, y = Accuracy, fill = Conjunto)) +
  geom_col(position = "dodge", width = 0.5) +
  geom_text(aes(label = paste0(Accuracy, "%")),
    position = position_dodge(width = 0.5),
    vjust = -0.5, fontface = "bold", size = 3.5) +
  ylim(0, 80) +
  scale_fill_manual(values = c("Entrenamiento" = "#1565C0",
    "Prueba" = "#E15759")) +
  labs(title = "Análisis de Sobreajuste - Accuracy Train vs Test",
```

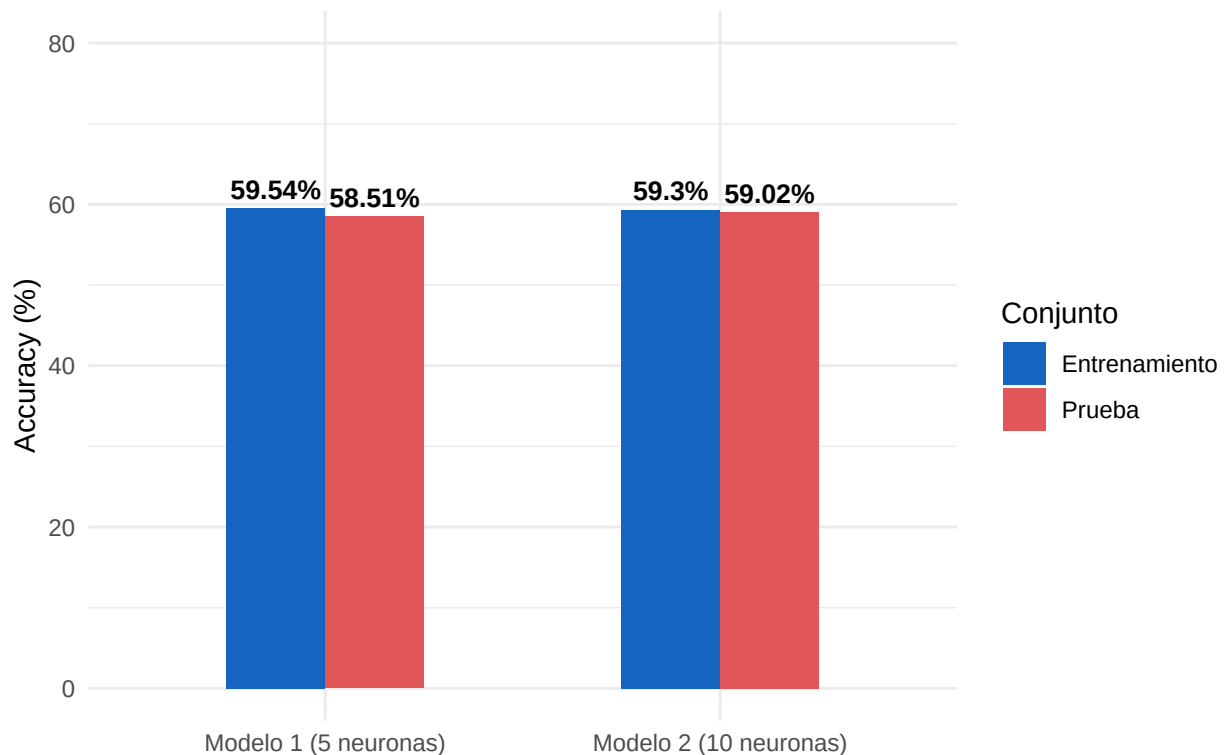
```

subtitle = "Una diferencia pequeña indica ausencia de sobreajuste",
x = NULL, y = "Accuracy (%)", fill = "Conjunto") +
theme_minimal()

```

Análisis de Sobreajuste – Accuracy Train vs Test

Una diferencia pequeña indica ausencia de sobreajuste



El análisis de sobreajuste confirma que ninguno de los dos modelos presenta sobreajuste significativo. El Modelo 1 obtuvo 59.54% en entrenamiento y 58.51% en prueba, una diferencia de apenas 1.03 puntos porcentuales. El Modelo 2 mostró la brecha más pequeña: 59.30% en entrenamiento y 59.02% en prueba, una diferencia de solo 0.28 puntos porcentuales.

8. Para el modelo elegido de clasificación tunea los parámetros y discuta si puede mejorar todavía el modelo sin llegar a sobre ajustarlo.

```

grid_tuning <- expand.grid(
  size = c(5, 10, 15, 20),
  decay = c(0.001, 0.01, 0.1, 0.5)
)

ctrl_tuning <- trainControl(method = "cv",
  number = 10,
  verbose = FALSE)

```

```

set.seed(123)
modelo_tuned <- train(precio_cat ~ .,
                      data      = train_rna_m,
                      method    = "nnet",
                      trControl = ctrl_tuning,
                      tuneGrid  = grid_tuning,
                      trace     = FALSE,
                      maxit     = 300)

cat("=== Resultados del Tuning ===\n")

```

```
## === Resultados del Tuning ===
```

```
print(modelo_tuned)
```

```

## Neural Network
##
## 5000 samples
##    5 predictor
##    3 classes: 'barata', 'cara', 'media'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 4499, 4500, 4500, 4500, 4500, 4500, ...
## Resampling results across tuning parameters:
##
##   size  decay  Accuracy  Kappa
##   5     0.001  0.5784021  0.3666397
##   5     0.010  0.5802001  0.3687402
##   5     0.100  0.5817989  0.3716818
##   5     0.500  0.5844025  0.3760754
##  10     0.001  0.5840001  0.3751731
##  10     0.010  0.5799997  0.3686652
##  10     0.100  0.5826057  0.3729532
##  10     0.500  0.5826045  0.3732805
##  15     0.001  0.5812013  0.3709469
##  15     0.010  0.5834013  0.3740170
##  15     0.100  0.5890001  0.3824436
##  15     0.500  0.5829981  0.3739533
##  20     0.001  0.5831973  0.3738659
##  20     0.010  0.5800009  0.3689039
##  20     0.100  0.5771953  0.3644141
##  20     0.500  0.5792037  0.3679470
##
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were size = 15 and decay = 0.1.

```

```
cat("\n=== Mejor combinación de parámetros ===\n")
```

```

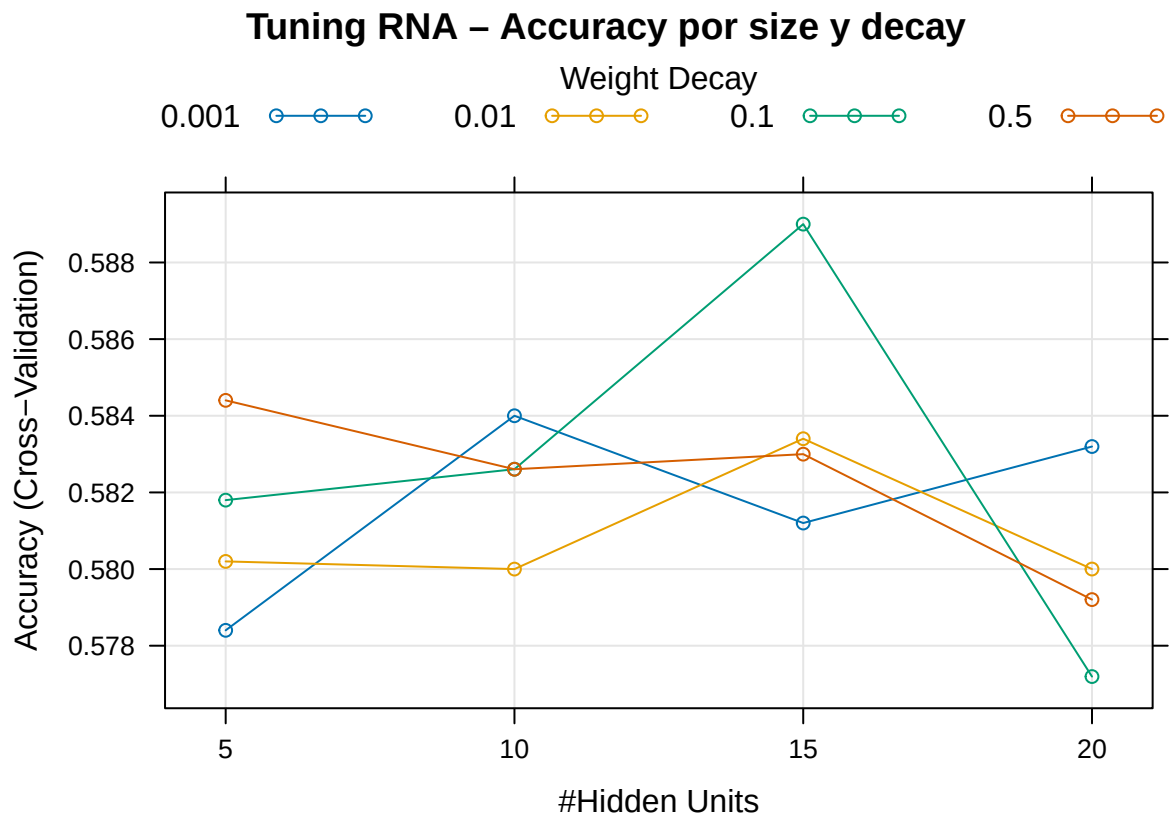
##
## === Mejor combinación de parámetros ===

```

```
print(modelo_tuned$bestTune)
```

```
##      size decay  
## 11      15  0.1
```

```
plot(modelo_tuned,  
      main = "Tuning RNA - Accuracy por size y decay")
```



El proceso de tuning exploró 16 combinaciones de parámetros mediante validación cruzada de 10 pliegues. La mejor combinación encontrada fue size=15 y decay=0.1, con un Accuracy CV de 58.90% y Kappa de 0.3824, superando al Modelo 2 original (size=10, decay=0.1) que obtuvo 58.48% y Kappa de 0.3800 en CV.

La mejora obtenida con el tuning es real pero modesta: apenas 0.42 puntos porcentuales de Accuracy sobre el modelo original. Esto sugiere que la arquitectura de red neuronal está cerca de su techo de desempeño para este conjunto de datos, y que la limitación principal no es la topología sino las variables predictoras disponibles. Para mejorar sustancialmente el modelo sería necesario incorporar nuevas variables como ubicación, tipo de propiedad o características del anfitrión, más que seguir ajustando size y decay.